From Stored-Program to Intent-Conditioned Computation (IIC): Rethinking the Paradigm for AI-Driven Systems

HUIWEN HAN, Lenovo China, China

For nearly a century, computing systems have been governed by the stored-program paradigm, in which explicit programs operate deterministically over data [8]. This assumption underpins core software engineering practices such as debugging, testing, and verification. However, modern AI-driven systems built on foundation models increasingly violate these assumptions: system behavior emerges from interactions among latent model capabilities, runtime intentions, retrieved knowledge, and contextual signals rather than static code alone [3].

This article introduces **Intent-Conditioned Computation (IIC)**, a conceptual computation paradigm in which execution is governed by dynamically injected semantic intentions that condition latent computational capabilities. We argue that prompts, policies, and control signals act as first-class execution parameters, fundamentally altering the meaning of "program," "input," and "correctness."

We present a high-level architectural decomposition of IIC into five functional roles: *Model Capabilities*, *Control Intentions*, *Externalized Knowledge*, *Deterministic Code*, and *Data*. This abstraction explains why behavior can change without code modification, why execution is probabilistic yet purposeful, and why classical debugging metaphors no longer suffice.

This work is part of the broader **AAI (Architectures of Autonomous Intelligence)** research program and aims to provide a shared conceptual foundation for reasoning about AI-native software systems in research and practice.

Additional Key Words and Phrases: Intent-Conditioned Computation, Foundation Models, Human-AI Interaction, Alignment, AI Safety

1 Motivation: Why Stored-Program Assumptions No Longer Hold

The stored-program concept, formalized by von Neumann, assumes a clear separation between program and data, with execution defined as the deterministic application of the former to the latter [8]. Under this model, identical code and input yield identical outputs, enabling step-wise debugging, regression testing, and formal verification.

These assumptions support classical notions of modular reasoning and correctness. Foundational software engineering principles such as information hiding and separation of concerns rely on the predictability and locality of program behavior [6].

AI-driven systems based on foundation models systematically violate these assumptions. Their observable behavior is no longer determined solely by explicit code, but emerges from interactions among latent model representations, runtime control signals, contextual knowledge, and stochastic inference mechanisms [3].

1.1 Observed Behavioral Phenomena

Three recurring phenomena illustrate the breakdown of stored-program assumptions:

Table 1. Behavioral phenomena that violate classical stored-program assumptions.

Phenomenon	Description
Behavioral plasticity	Minor changes in prompts or instructions lead to qualitatively different system behavior, without any modification to underlying code.
Context dependence	Execution outcomes depend on retrieved documents, examples, or interaction history, which are not part of the static program.
Probabilistic execution	Identical inputs may yield multiple valid outputs, reflecting a distribution over behaviors rather than a single result.

1.2 Illustrative Example

Consider a language model queried with identical input data but different intentions: *"Summarize this document"* versus *"Explain this document to a non-expert audience."* The system produces divergent outputs, despite executing the same code on the same data. The differentiating factor is neither algorithm nor dataset, but the injected *intent*.

This behavior cannot be adequately explained using stored-program terminology. Treating prompts as "data" or "configuration" fails to capture their role in shaping execution semantics.

Author's Contact Information: Huiwen Han, Lenovo China, Technology and Solution Delivery, Beijing, China, hanhuiwen@gmail.com.

1.3 Conceptual Mismatch

Table 2. Conceptual mismatch between stored-program execution and AI-driven systems.

Aspect	Stored-Program	AI-Driven Systems
Program	Explicit, static code	Latent, distributed in model parameters
Input	Data only	Data + intent + context
Execution	Deterministic	Probabilistic, intent-sensitive
Correctness	Exact output match	Alignment with intended behavior

This mismatch reflects a deeper limitation of program-centric computation models. As Backus argued, describing computation purely as state-transforming programs obscures higher-level semantic structure and behavior [2].

These observations motivate the need for a revised computation paradigm that treats intent and context as first-class execution elements. Intent-Conditioned Computation addresses this gap.

2 Intent-Conditioned Computation (IIC)

Intent-Conditioned Computation (IIC) defines execution as the activation of latent computational capabilities conditioned on runtime intentions and contextual signals. Rather than viewing computation as the execution of a fixed program, IIC models computation as a dynamic process in which behavior emerges from semantic control layered over probabilistic inference.

3 Intent-Conditioned Computation (IIC)

Intent-Conditioned Computation (IIC) defines execution as the activation of latent computational capabilities conditioned on runtime intentions and contextual signals. Rather than viewing computation as the execution of a fixed program, IIC models computation as a dynamic process in which behavior emerges from semantic control layered over probabilistic inference.

This formulation reflects empirical properties of foundation-model-based systems, in which a single trained model can support diverse tasks depending on how it is prompted and contextualized [3].

3.1 Core Definition

> *In Intent-Conditioned Computation, system behavior is determined by the interaction between latent capabilities and explicitly specified runtime intentions, augmented by contextual knowledge and constrained by deterministic mechanisms.*

In this model, prompts, policies, and control directives are not auxiliary inputs but execution-defining elements.

3.2 Architectural Decomposition

IIC systems can be decomposed into five functional roles:

Table 3. Functional components of Intent-Conditioned Computation.

Component	Role in Computation
Model Capabilities	Latent behaviors encoded in trained models, defining the space of possible actions.
Control Intentions	Runtime semantic directives that prioritize, constrain, or shape how capabilities are activated.
Externalized Knowledge	Retrieved documents, examples, or memories that augment context beyond the model's parameters.
Deterministic Code	Traditional software mechanisms enforcing constraints, validation, safety, and integration.
Data	Inputs and outputs serving as the observable interface to computation.

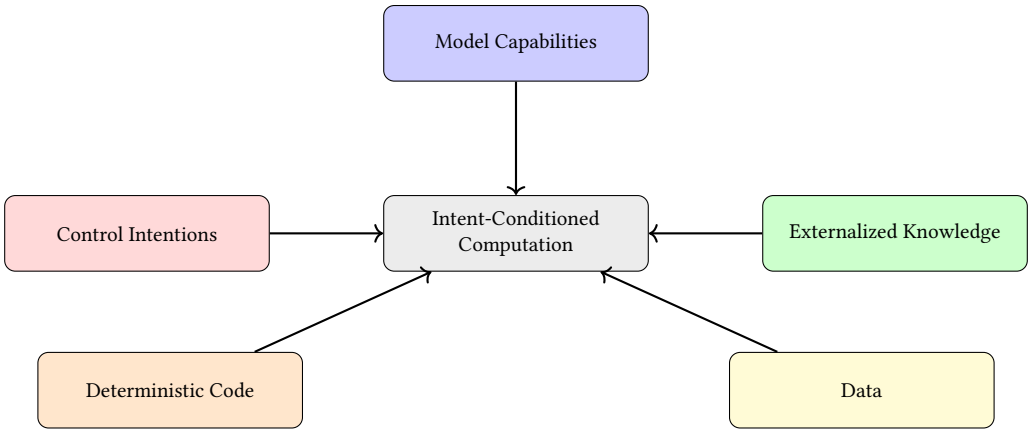

Fig. 1. Architectural roles in Intent-Conditioned Computation. Edges denote conceptual dependency rather than execution order.

This decomposition generalizes classical modularity principles from deterministic procedures to probabilistic and intent-sensitive behavior, extending the spirit of information hiding to AI-native systems [6].

3.3 Structural View

3.4 Implications of the IIC Model

The IIC paradigm leads to several immediate implications:

- Behavior can change without code modification.
- Correctness must be evaluated relative to intent, not exact output.
- Execution traces require intent-aware interpretation.

These properties distinguish IIC as a computation paradigm rather than an implementation detail, laying the foundation for revised software engineering, evaluation, and governance practices discussed in subsequent sections.

4 Key Insights and Paradigm Shift

Intent-Conditioned Computation (IIC) is not merely a description of how foundation-model systems behave in practice [4]. It represents a conceptual shift in how computation itself is defined, observed, and evaluated. This section distills three core insights that distinguish IIC from the classical stored-program paradigm [8].

4.1 Insight I: Programs Are No Longer the Sole Carriers of Behavior

In stored-program systems, behavior is localized in explicit code. Inputs select execution paths, but do not redefine the semantics of execution. In contrast, IIC systems distribute behavioral definition across latent model parameters and runtime control intentions, a pattern now common in large language model deployments [7].

Under IIC:

- Code specifies *mechanisms* and constraints.
- Model parameters encode a space of *potential behaviors*.
- Control intentions select and shape behaviors at runtime through prompts, policies, and semantic directives [9].

As a result, no single artifact can be identified as *the program*. Behavior emerges from the interaction of multiple representational layers, some of which are opaque, probabilistic, and learned rather than written.

This explains why modifying an intent prompt can alter system behavior as fundamentally as rewriting a function in classical software, even though no executable code changes.

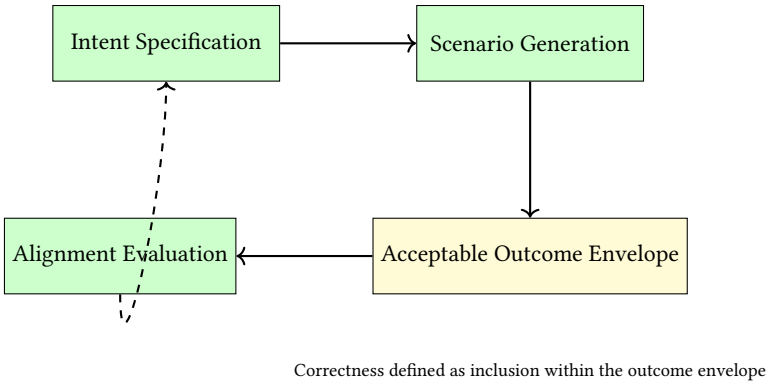

Fig. 2. Intent-aligned correctness in IIC systems. Rather than validating a single deterministic output, systems are evaluated against an acceptable envelope of behaviors derived from explicit intent specifications.

4.2 Insight II: Execution Is Intent-Sensitive Rather Than Deterministic

Classical execution semantics assume that determinism is the norm and nondeterminism is an exception. IIC reverses this assumption.

Execution in IIC systems is best understood as sampling from a behavioral distribution constrained by intent and context, a property repeatedly observed in generative AI systems [4]. Determinism, when required, is reintroduced selectively through deterministic code, validation layers, or post-processing.

Table 4. Reinterpreting execution semantics under IIC.

Aspect	Stored-Program	IIC
Execution model	Fixed state transition	Intent-conditioned sampling
Role of randomness	Exceptional	Fundamental
Control mechanism	Control flow	Semantic intent
Failure mode	Incorrect output	Misaligned behavior

This reframing clarifies why repeated executions of the same IIC system may yield different outputs without indicating a fault. Variation is not noise; it is an intrinsic property of intent-conditioned execution.

4.3 Insight III: Correctness Becomes Intent Alignment

In classical software engineering, correctness is defined as conformance to a specification, typically expressed as exact input–output relationships. Such definitions are increasingly inadequate for AI systems, where multiple outputs may all satisfy the same user intent [1].

IIC replaces exact correctness with *intent alignment*: the degree to which observed behavior falls within an acceptable range of outcomes implied by a given intent.

This shift implies that testing becomes scenario-based, debugging becomes intent-tracing, and assurance depends on coverage of behavioral space rather than exhaustive enumeration of outputs.

4.4 A Paradigm Shift in Computation

Taken together, these insights establish IIC as a distinct computation paradigm:

- From *program execution* to *capability activation.*
- From *deterministic correctness* to *intent alignment.*
- From *static control flow* to *semantic runtime control.*

IIC does not replace stored-program computation; rather, it subsumes it as a special case in which intent is fixed, context is minimal, and probabilistic variation is suppressed.

5 Implications for Practice

Intent-Conditioned Computation (IIC) fundamentally changes how AI-driven systems should be engineered, evaluated, and governed. This section translates the conceptual insights of IIC into concrete implications for software engineering practice, AI system design, and institutional governance.

5.1 Software Engineering: From Output Correctness to Intent Alignment

In IIC systems, testing and debugging can no longer rely on fixed input–output equivalence. Instead, engineers must reason about whether observed behavior is *aligned* with the intended purpose of the system, a shift already advocated in human-centered AI engineering research [1].

Key shifts in practice include:

- Replacing unit tests with *scenario-based evaluation* covering representative contexts and variations.
- Documenting *intended behaviors* rather than exact outputs.
- Treating behavioral variance as expected, while bounding it through outcome envelopes.

Example. A code completion system may generate multiple valid implementations for the same input. Correctness is determined by whether generated code satisfies the intended constraints (e.g., safety, performance, readability), not whether it matches a reference output.

This reframing positions testing as an exercise in intent specification and coverage, rather than exhaustive enumeration.

5.2 AI System Design: Architecting for Intent Modulation

Designing systems under IIC requires explicit architectural support for runtime intent injection and modulation. Intent should be treated as a first-class control signal, not an implicit side effect of user input, consistent with emerging prompt- and policy-driven system designs [9].

Table 5. Design considerations for Intent-Conditioned Computation systems.

Design Element	Engineering Practice	Benefit
Model capabilities	Modular capability abstraction	Reusable behaviors across tasks
Control intentions	Explicit intent interfaces	Dynamic behavior adaptation
Knowledge integration	Context retrieval pipelines	Improved relevance and grounding
Deterministic code	Validation and constraint layers	Safety and reliability guarantees
Data handling	Input/output monitoring	Observability and audit support

Separating *capabilities* from *intent* allows the same system to support diverse use cases without retraining or redeployment. Deterministic components remain essential, but their role shifts toward bounding and validating probabilistic behavior rather than replacing it.

5.3 Governance and Accountability: Making IIC Auditable

Governance challenges in AI systems are often framed as problems of model transparency. IIC reframes governance as a problem of *intent traceability*, an approach aligned with safety-case and assurance-based governance models [5].

For regulated or safety-critical deployments, organizations must be able to answer: *What was the system trying to do at the time of execution?*

Governance mechanisms under IIC include:

- Logging runtime intents alongside inputs and outputs.
- Recording contextual knowledge sources used during execution.
- Maintaining auditable links between intent specifications and observed system behavior.

These practices enable post hoc analysis without requiring full interpretability of model internals. Accountability is achieved through reconstructing the intent–context–outcome chain, rather than explaining individual model decisions.

5.4 Operational Roadmap: From Engineering to Institutions

Operationalizing IIC is a staged process that reflects organizational maturity rather than model capability.

- **Level II: Operational Evaluation.** Organizations develop intent-aligned metrics, scenario coverage benchmarks, and robustness tests for probabilistic behavior.
- **Level III: Governance Integration.** Evaluation practices are institutionalized through auditable decision logs, control frameworks, and standardized guidelines.

This roadmap clarifies that advancing IIC in practice is not solely a research challenge. It requires coordinated progress across engineering, process design, and institutional governance.

5.5 Summary

- Software engineering under IIC shifts from deterministic correctness to intent alignment.
- AI system design benefits from modular capabilities and explicit intent control.
- Governance focuses on intent traceability rather than full model transparency.

Together, these implications position IIC as a practical framework for building, evaluating, and governing AI-native systems at scale.

References

[1] Saleema Amershi, Daniel S. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. T. Iqbal, P. N. Bennett, M. Williams, et al. 2019. Guidelines for Human-AI Interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. ACM, ACM, New York, NY, USA, 1–13.

[2] John Backus. 1978. Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs. *Commun. ACM* 21, 8 (1978), 613–641.

[3] Rishi Bommasani et al. 2021. *On the Opportunities and Risks of Foundation Models*. Technical Report. Stanford Center for Research on Foundation Models.

[4] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simranjeet Arora, Shantanu von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. 2021. On the Opportunities and Risks of Foundation Models. *arXiv preprint arXiv:2108.07258* abs/2108.07258 (2021), 1–76.

[5] NASA. 2011. *NASA Safety-Critical System Guidelines*. Technical Report NASA-STD-8719.13B. NASA, Washington, D.C.

[6] David L. Parnas. 1972. On the Criteria to Be Used in Decomposing Systems into Modules. *Commun. ACM* 15, 12 (1972), 1053–1058.

[7] Jack Rae, Sebastian Borgeaud, Tiffany Cai, Katie Millican, Jordan Hoffmann, Helen Song, Adam Roberts, Amanda Askell, Bas van Merriënboer, Helen Ring, et al. 2021. Scaling Language Models: Methods, Analysis & Insights from Training Gopher. *arXiv preprint arXiv:2112.11446* abs/2112.11446 (2021), 1–50.

[8] John von Neumann. 1945. *First Draft of a Report on the EDVAC*. Technical Report. University of Pennsylvania.

[9] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, Denny Zhou, and Ed Chi. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *arXiv preprint arXiv:2201.11903* abs/2201.11903 (2022), 1–20.